Integrante 1: Núcleo, Configuração e Inicialização

FELIPE

Foco na estrutura base, banco de dados e script de testes.

- **Arquivos/Pastas sob responsabilidade:**
  - `app.py`, `config.py`, `requirements.txt`, `README.md`
  - `dados/` (criação e configuração inicial do SQLite)
  - `financas/servicos/inicializacao.py`
  - `scripts/popular_exemplo.py` (essencial para gerar os dados de teste para a equipe)

Integrante 2: Modelos, Camada de Dados e Repositórios

MURILLO

Foco na modelagem de dados e nas operações de persistência.

- **Arquivos/Pastas sob responsabilidade:**
  - `financas/modelos/` (todas as *dataclasses*: `transacao.py`, `categoria.py`, `recorrencia.py`, `meta.py`, `investimento.py`, `previsao.py`, `relatorio.py`)
  - `financas/repositorios/` (`banco.py`, `transacao_repo.py`, `meta_repo.py` e demais repositórios)

Integrante 3: Serviços de Negócio (Transações, Metas e Previsões)

SOPHIA

Foco na lógica financeira principal da aplicação.

- **Arquivos/Pastas sob responsabilidade:**
  - `financas/servicos/resumo_service.py`
  - `financas/servicos/meta_service.py`
  - `financas/servicos/investimento_service.py`
  - `financas/servicos/previsao_service.py`
  - `financas/utils/` (`dinheiro.py`, `datas.py`, `validacao.py`)

Integrante 4: Relatórios, PDF e Testes Automatizados

HEITOR

Foco na exportação de documentos e garantia de qualidade.

- **Arquivos/Pastas sob responsabilidade:**
  - `financas/servicos/relatorio_service.py`
  - `financas/servicos/pdf_service.py` (geração dos relatórios mensais em PDF)
  - `tests/` (`test_investimento.py`, `test_meta.py` e demais testes unitários)
  - Pasta `relatorios/`

Integrante 5: Interface, Rotas (Blueprints), Templates e Front-end

VITOR

Foco na experiência do usuário, telas e gráficos.

- **Arquivos/Pastas sob responsabilidade:**
  - `financas/rotas/` (`painel.py`, `lancamentos.py`, `metas.py`, `investimentos.py`, `relatorios.py`, `api.py`)
  - `templates/` (`base.html`, `painel.html`, `lancamentos.html`, etc.)
  - `static/css/style.css` e `static/js/` (`graficos.js` com Chart.js, `filtros.js`)

Dicas para a Equipe Trabalhar Junta:

1. **Ordem de Integração:** O **Integrante 1** e o **Integrante 2** devem começar primeiro para entregar a base do banco e os modelos. Assim que os modelos estiverem prontos, os Integrantes 3 e 4 conseguem construir os serviços.
2. **O Integrante 5** pode criar os templates HTML e a estrutura visual usando dados fictícios (mocks) enquanto os serviços do back-end não estão 100% prontos, integrando as rotas do Flask na reta final.